Gestión de rutas y directorios

Uno de los principales objetivos de Python, suele el desarrollo de scripts que automaticen tareas, para ello suele ser conveniente poder interaccionar con el sistema de directorios o con los comandos del sistema. Con ese objetivo existen una serie de librerias que se van a presentar a lo largo del siguiente documento tales como `os`, `shutil` o `pathlib`.

1 MANEJO DEL ÁRBOL DE DIRECTORIOS

Los problemas con los que habitualmente te vas a encontrar al intentar manejar el árbol de directorio son:

- Diferencias entre sistemas operativos (Windows / Linux / macOS)
- Separadores de ruta (`\()` y `\\`)
- Rutas relativas y absolutas
- Eliminación accidental de datos
- Permisos insuficientes
- Código poco legible o difícil de mantener

!!

Otro problema que suele suceder es no saber desde donde se está ejecutando el programa, para ello, existe un comando que te muestra cuál es el directorio actual:

```
import os
print(os.getcwd())
```

2 GESTIÓN DE RUTAS

En Python existen dos formas principales de trabajar con rutas:

- **`os.path`** es un submódulo del módulo `os` que proporciona funciones para manipular rutas como cadenas de texto.
- **`pathlib.Path`**: `Path` es una clase del módulo `pathlib` que representa una ruta como un objeto.

2.1 GESTIÓN DE RUTAS CON `OS.PATH`

La gestión de rutas con `os.path` viene vinculada al manejo de cadenas de texto y de la generación de las mismas a través de la función `os.path.join` que conformará la ruta al fichero o directorio objetivo indiferentemente del sistema operativo.

```
import os
ruta = os.path.join("logs", "2026", "enero")
```

Las operaciones con las que habitualmente te vas a encontrar son:

- Verificar si existe el directorio o fichero especificado en la ruta:

  ```
  os.path.exists(ruta)
  ```

- Verificar si la ruta es un fichero.

  ```
  os.path.isfile(ruta)
  ```

- Verificar si la ruta es un directorio.

  ```
  os.path.isdir(ruta)
  ```

- Obtener la ruta absoluta

  ```
  os.path.abspath(ruta)
  ```

- Obtener el nombre del archivo o directorio destino.

  ```
  os.path.basename(ruta)
  ```

- Obtener el nombre del directorio madre.

  ```
  os.path.dirname(ruta)
  ```

- Devolver una `tuple` conformada por el directorio madre y el nombre del directorio o archivo objetivos.

  ```
  os.path.split(ruta)
  ```

- Devolver una `tuple` conformada por el nombre y la extensión, en caso de que sea un fichero la segunda parte de la tupla estará vacía.

  ```
  os.path.splitext(ruta)
  ```

- Verificar si existe directorio o fichero.

  ```
  os.path.exists(ruta)
  ```

- Verificar si es una ruta absoluta

  ```
  os.path.isabs(ruta)
  ```

- Obtener el tamaño en bytes

```
os.path.getsize(ruta)
```

- Obtener la fecha de la última modificación en formato tiempo UNIX (`float`).

```
import datetime
datetime.date.fromtimestamp(os.path.getmtime(ruta))
```

- Obtener la fecha creación en formato tiempo UNIX (`float`).

```
import datetime
datetime.date.fromtimestamp(os.path.getctime(ruta))
```

- Verificar si dos rutas apuntan al mismo destino.

```
os.path.samefile(p1, p2)
```

2.2 GESTIÓN DE RUTAS CON PATH

El módulo `pathlib` incluye la clase `Path` y es una forma más moderna y recomendada para trabajar con rutas. Interpreta la ruta como un objeto que permite concatenar rutas con el operador `/`.

```
from pathlib import Path
ruta = Path("carpeta/archivo.txt")
ruta2 = Path("carpeta") / "archivo2.txt"
```

- Obtener ruta absoluta.

```
ruta.resolve()
```

- Obtener nombre del archivo con extensión.

```
ruta.name
```

- Obtener el nombre del archivo sin extensión.

```
ruta.stem
```

- Obtener la extensión.

```
ruta.suffix
```

- Obtener la lista extensiones.

```
ruta.suffixes
```

- Obtener el directorio madre.

```
ruta.parent
```

- Obtener todos los directorios madre.

```
ruta.parents
```

- Obtener la ruta como partes de una tupla

```
ruta.parts
```

- Verificar si existe

```
ruta.exists()
```

- Verificar si es un archivo.

```
ruta.is_file()
```

- Verificar si es un directorio.

```
ruta.is_dir()
```

- Verificar si es una ruta absoluta.

```
ruta.is_absolute()
```

- Crear un directorio.

```
ruta.mkdir()
```

- Crear un directorio de manera anidada.

```
ruta.mkdir(parents=True, exist_ok=True)
```

Este método acepta 3 posibles parámetros opcionales:

1. `mode` que por defecto tiene el valor `0o777` y es de tipo `int`. Especifica los permisos para crear el último directorio de la ruta.
2. `parents` especifica como el método lidia con los directorios que falten en la ruta. Hay dos opciones:
   - Con el valor por defecto `False` se genera un error `FileNotFoundError` cuando se encuentra que alguno de los directorios de la ruta no existe.

- Con `True` crea los directorios que falten.

3. `exist_ok` controla como lidiar con las rutas que sí existan. Las opciones son:
   - Con el valor por defecto `False` se lanza el error `FileExistsError` cuando la ruta ya existe.
   - Con el valor `True` no lanza el error, aunque suceda.

- Crear un archivo vacío.

```
ruta.touch()
```

- Eiminar un archivo

```
ruta.unlink()
```

- Eliminar un directorio vacío

```
ruta.rmdir()
```

- Renombrar o mover.

```
ruta.rename("archivito.txt")
```

Se ha renombrado el fichero de `archivo.txt` a `archivito.txt`

- Listar archivos y carpetas

```
for dir in ruta.iterdir():
    print(dir)
```

- Realizar una búsqueda.

```
for dir in ruta.glob("*.pdf"):
    print(dir)
```

- Obtener información del archivo.

```
ruta.stat()
```

3 EJECUCIÓN DE COMANDOS DEL SISTEMA OPERATIVO

En Python, puedes ejecutar comandos del sistema (como si los ejecutaras en la terminal) usando varios métodos.

3.1 OS.SYSTEM

El método más simple, pero no captura la salida del comando. Retorna 0 si el comando fue exitoso.

```
import os
os.system("touch file.txt")
```

3.2 SUBPROCESS.RUN

Este método permite ejecutar un comando con sus argumentos además de capturar la salida y posibles errores.

```
import subprocess
resultado = subprocess.run(["ls", "-l"], capture_output=True, text=True)
print(f"Código de salida: {resultado.returncode})
print(f"Salida del comando:\n{resultado.stdout})
print("Errores:\n{resultado.stderr)
```

4 OPERACIONES SOBRE ARCHIVOS Y DIRECTORIOS

El módulo `shutil` en Python sirve para realizar operaciones de alto nivel sobre archivos y directorios, especialmente copiar, mover, eliminar y comprimir contenido del sistema de archivos.

- Copiar archivos, copiando contenido y permisos:

```
import shutil
shutil.copy("origen.txt", "destino.txt")
```

- Copiar archivos y metadatos:

```
shutil.copy2("origen.txt", "destino.txt")
```

- Copiar directorios completos

```
shutil.copytree("carpeta_origen", "carpeta_destino")
```

- Mover o renombrar:

```
shutil.move("archivo.txt", "nueva_carpeta/")
```

- Comprobar uso de disco:

```
uso = shutil.disk_usage("/")
print(uso.total, uso.used, uso.free)
```

- Crear archivos comprimidos:

```
shutil.make_archive("backup", "zip", "carpeta")
shutil.unpack_archive("backup.zip", "destino")
```